

Regression Bug Classification Guideline

*A Multi-Artifact Classification Guideline for Identifying
Regression Bugs in Post-Release GitHub Issue Reports*

Mastodon v4.3 Release Family — Pilot Study

1. PURPOSE

This guideline supports the manual classification of GitHub issue reports from the Mastodon v4.3 release family. It is designed to help reviewers determine, first, whether each issue describes a software defect, and second, whether that defect is a regression bug — a failure in behavior that previously worked correctly but became broken, degraded, or incorrect after a software change or release.

The guideline follows a **two-stage classification process** and is applied exclusively to the review of GitHub issue pages and their linked artifacts — issue comments, pull requests, commits, and release notes. It does not involve software testing or code execution.

Stage	Classification Task
Stage A	Is the issue a Bug, Not a Bug, or Unknown?
Stage B	If Bug: is it a Regression Bug, Other Bug, or Unknown?

2. DEFINITIONS

All classification labels are grounded in established software engineering definitions. The following definitions apply throughout the guideline and must be consistently applied by all reviewers.

2.1 Stage A Labels

Bug A Bug is an issue reporting a software defect, failure, broken behavior, incorrect behavior, or unexpected behavior in the system. This includes issues documenting corrective maintenance tasks that require semantic changes to source code [1, 3].

Not a Bug A Not a Bug issue does not report a software defect. It refers instead to a feature request, enhancement, documentation change, refactoring-related request, support need, or similar non-defect item. These correspond to perfective and adaptive maintenance, refactoring, discussions, or requests for help — activities unrelated to errors in the code [1, 3].

Unknown An Unknown issue is vague, incomplete, or ambiguous for the reviewer to determine whether it describes a software defect or a non-defect item. Assign Unknown only when there is genuinely insufficient evidence to support either a Bug or Not a Bug decision.

2.2 Stage B Labels

Regression Bug A Regression Bug is a bug in which a feature or behavior worked correctly before a software change or release, but became broken, degraded, or incorrect afterward [2]. Regression bugs are unintended consequences of program changes — they occur whenever software functionality that previously worked no longer works as planned [5]. This includes regressions caused by a commit fixing an existing bug, a new feature implementation, a dependency update, a configuration change, or a UI modification [2, 4].

Other Bug An Other Bug is a confirmed software defect that is not a regression bug. This includes defects in newly introduced features that never previously worked correctly, and defects in existing features whose problem already existed before the focal release. The key distinction from a Regression Bug is the absence of evidence of a prior correct state.

Unknown Assign Unknown in Stage B when the available issue page and related artifacts do not provide enough reliable evidence to determine whether the bug is a regression bug or not. This applies to cases where prior behavior cannot be established, where the introducing change cannot be identified, or where evidence is contradictory.

3. EVIDENCE SOURCES

Regression evidence is rarely contained in a single artifact. It is typically distributed across the issue report, the discussion thread, and linked development artifacts. Reviewers must inspect all available and relevant artifact types before making a classification decision. Evidence sources are reviewed in the following order:

Level	Artifact	What to look for
1	Issue title and body	Version references, behavioral comparisons, temporal language, labels, expected vs. actual behavior, reproduction steps.
2	Issue discussion	Developer or maintainer confirmation of prior behavior; technical diagnosis of the introducing change; links to related issues or PRs.
3	Linked pull requests	PR title and description mentioning a fix or revert; PR discussion identifying the regression-inducing change; merge confirmation.
4	Commits and code changes	Commit messages referencing the issue or a fix; code diffs showing restoration of prior behavior; revert commits.
5	Release notes / changelogs	Official project documentation confirming a fix was shipped in a subsequent release; changelog entries referencing the issue or PR.

4. CLASSIFICATION DECISION FLOW

The following decision flow summarizes the classification process. Follow each step in order. Do not skip steps or move to Stage B without completing Stage A.

1	Read the issue title, body, and all comments.
	→ Clearly a defect? → Assign Bug and proceed to Step 2. → Clearly not a defect? → Assign Not a Bug and stop. → Insufficient evidence? → Assign Unknown and stop.
2a	Inspect issue title, body, and comments for regression signals.
	Look for: (1) prior working behavior, (2) post-change failure language, (3) evidence that feature is new. <i>Note what you find and proceed to 2b.</i>
2b	Inspect linked artifacts (pull requests, commits, release notes).
	Follow artifact links from the issue. Look for repair evidence, introducing change identification, or release confirmation. <i>Not all artifacts will be present. Assess available evidence holistically.</i>
2c	Assign the final Stage B label.
	Prior correct state + post-change failure confirmed? → Regression Bug Feature is new, or problem predates the release? → Other Bug Insufficient evidence across all artifacts? → Unknown
2d	Record the label, evidence sources, and a brief rationale in the spreadsheet.

5. REFERENCES

- [1] K. Herzig, S. Just, and A. Zeller, "It's not a bug, it's a feature: how misclassification impacts bug prediction," in Proc. ICSE, IEEE, 2013, pp. 392–401.
- [2] G. Xiao, Z. Zheng, B. Jiang, and Y. Sui, "An empirical study of regression bug chains in Linux," IEEE Trans. Reliability, vol. 69, no. 2, pp. 558–570, 2019.
- [3] G. Antoniol, K. Ayari, M. Di Penta, F. Khomh, and Y.-G. Guéhéneuc, "Is it a bug or an enhancement? A text-based approach to classify change requests," in Proc. CASCON, 2008, pp. 304–318.
- [4] M. Khattar, Y. Lamba, and A. Sureka, "Sarathi: Characterization study on regression bugs and identification of regression bug inducing changes," in Proc. ISEC, ACM, 2015, pp. 50–59.
- [5] L. Braz, E. Fregnan, V. Arora, and A. Bacchelli, "An exploratory study on regression vulnerabilities," in Proc. ESEM, ACM, 2022, pp. 12–22.